

Exercise 1: Implementing the Singleton Pattern

Scenario: Ensure that a logging utility class has only one instance throughout the application lifecycle.

1. Create a New Java Project

Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class (Logger.java)

```
public class Logger { private static Logger instance; private Logger() { // Private constructor } public static Logger getInstance() { if (instance == null) { instance = new Logger(); } return instance; } public void log(String message) { System.out.println("LOG: " + message); } }
```

3. Implement the Singleton Pattern

The Logger class follows the Singleton pattern because:

The constructor is private, preventing external object creation. A private static instance variable stores the single object. The public static getInstance() method provides global access to the single instance.

4. Test the Singleton Implementation (SingletonTest.java)

```
public class SingletonTest { public static void main(String[] args) { Logger logger1 = Logger.getInstance(); Logger logger2 = Logger.getInstance(); logger1.log("First message"); logger2.log("Second message"); if (logger1 == logger2) { System.out.println("Only one Logger instance exists."); } else { System.out.println("Different Logger instances exist."); } } }
```

Expected Output

LOG: First message LOG: Second message Only one Logger instance exists.